

Observability & Evals

Better Harness: A Recipe for Harness Hill-Climbing with Evals



Vivek Trivedy

April 8, 2026

8 min



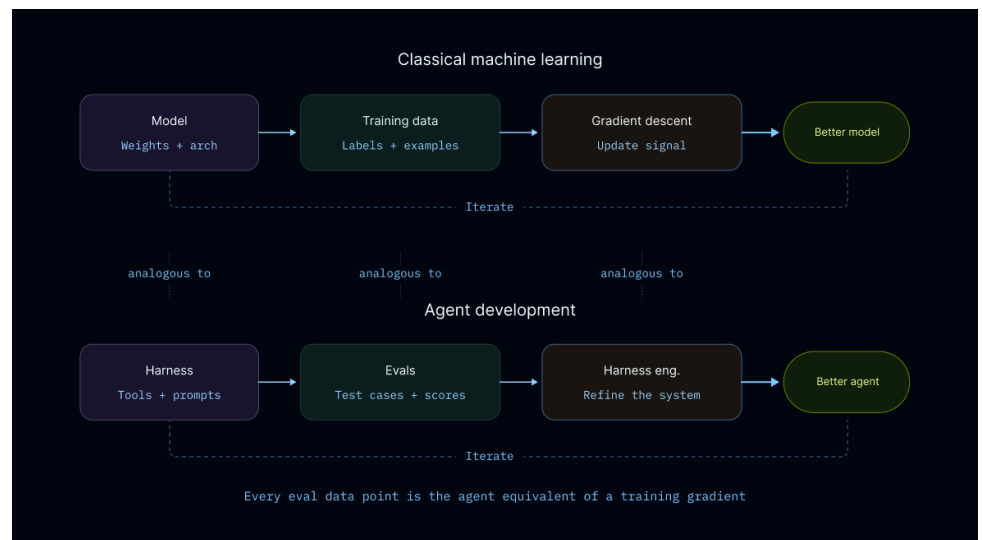
Key Takeaways

TL;DR: We can build better agents by building better harnesses. But to autonomously build a “better” harness, we need a strong learning signal to “hill-climb” on. We share how we use evals as that signal, plus design

decisions that help our agent generalize instead of overfit. Better-Harness is a system for iteratively sourcing and improving your harness with evals.

Evals are training data for agents

In classical machine learning, training data guides the model's learning process. Each training example contributes a gradient that updates the model's weights toward "correctness." We have a similar learning loop for agents.



Evals encode the behavior we want our agent to exhibit in production. They're the "training data" for harness engineering. Each eval case contributes a signal like "did the agent take the right action" or "produce the right outcome?" That signal guides the next proposed edit to the harness.

The same rigor and care we put into data quality and curation for model training should also go into eval design. We discuss the importance of data quality in a previous post, [how we build evals for Deep Agents](#).

There's some great recent work that formalize the steps to optimize harnesses including [Meta-Harness](#) from Stanford and [Auto-Harness](#) from DeepMind. We also previously shared a [Harness Improvement Loop to hill-climb Terminal Bench 2.0](#) by just tweaking the harness layer. We think there's great future work to be done around the update algorithm itself, but harness improvement is a compound system that goes beyond the update algorithm which is what we talk about here.

Better-Harness is a take on compound systems engineering.

data sourcing → experiment design → optimization → review & acceptance

So we include practical details that go alongside the update loop such as how we source evals in the first place, how we design against overfitting, store traces over time, and manually review updates to sanity check anything we ship to production.

Sourcing good evals

Evals are the foundation that power the harness hill-climbing process. Here are the practical ways we source, curate, and use them.

Hand-curated. For any given task, the team manually writes examples that capture what we think the agent should do in production. These are often high value, but difficult to generate at scale.

Production traces. Every agent interaction generates a trace where failures become eval cases. Mining traces for eval material is the leverage, high-throughput way to improve evals over time. Even before running an agent over evals, often a team dogfooding our agent will report errors directly in Slack with a Trace link. We recommend dogfooding agents and directly sharing feedback for everyone to see, it helps build shared knowledge of agent behavior.

External datasets. These datasets are useful but need to be manually curated to make sure the test cases used to improve the agent reflect desired behaviors. Often each task is adjusted to make sure they measure the important behavior.

Tag everything. Every eval gets tagged to behavioral categories: "tool selection," "multi-step reasoning," etc. Tags enable meaningful holdout sets and targeted experiments. It also saves a lot of money because we can run subsets of evals.

Building learning systems that generalize

The ideal outcome for any learning system is **generalization**. We give an input signal that captures the distribution of behaviors we want in the wild. The system fits to it and then "just works" on new inputs it's never seen.

The obvious problem: We don't have unlimited data.

The fix: Encode important behaviors into curated evals. Quality > quantity, a small set of well-tagged evals covering the behaviors you care about beats thousands of noisy but high-coverage evals.

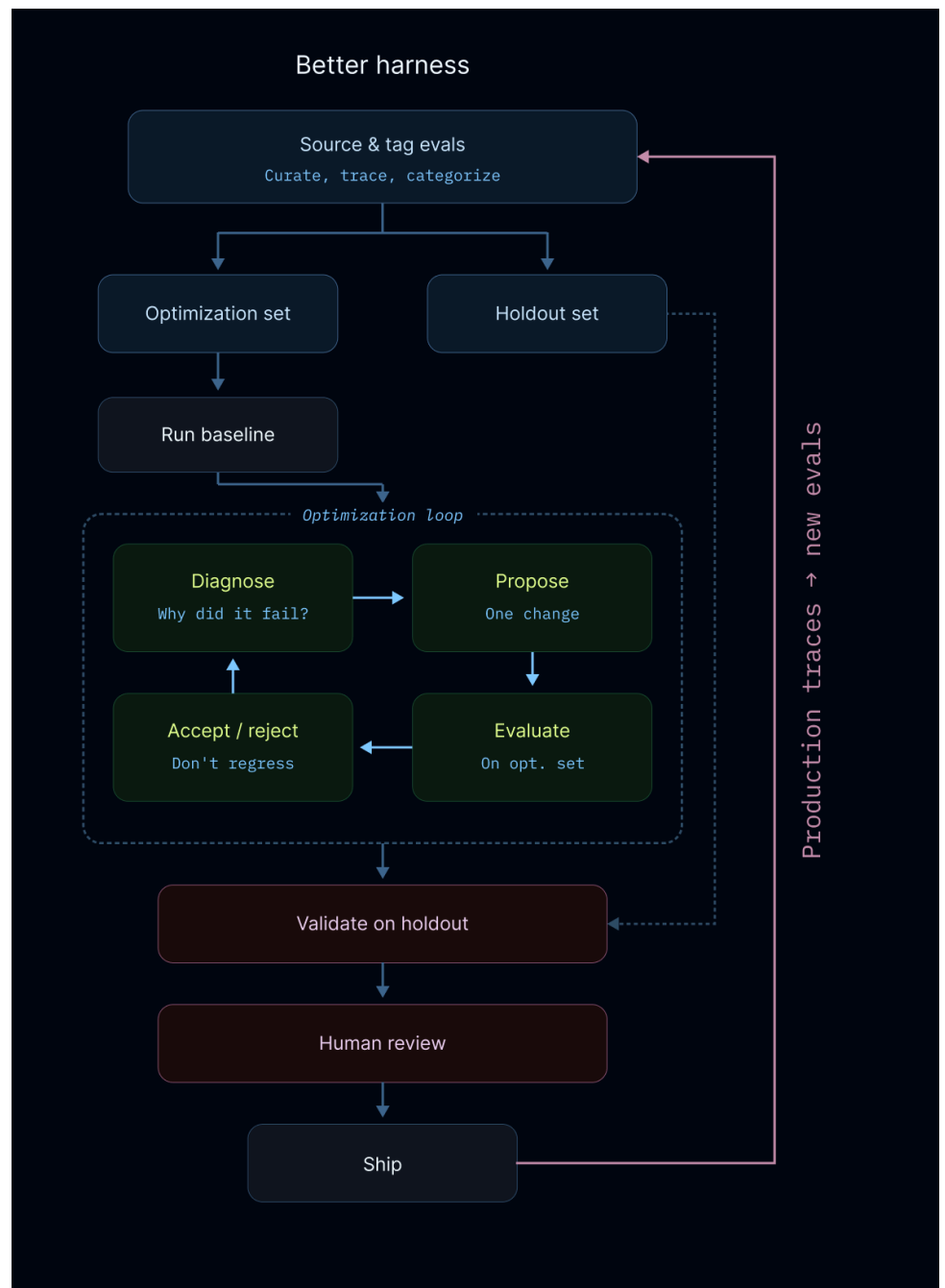
The subtle problem → agents are famous cheaters: Any learning system is prone to reward hacking where the agent overfits its structure to make the existing evals pass that it can see. This makes sense because the loop just wants to "make number go up" and doesn't know about generalization. We prompt to avoid overfitting but it isn't perfect.

The fix: Holdout sets become a proxy for true generalization. We've seen approaches that we pair with human review as a second signal and we get semi-automated systems can improve scores while avoiding behaviors we don't want in prod.

Better-Harness: a recipe for hill climbing your harness

We created a scaffold for autonomously improving our harness using evals as a signal in each step. A research version is open sourced [here](#), here are the main steps:

1. **Source and tag evals.** This is a mix of hand-writing evals, mining them from production traces, and using/adapting external datasets. We tag each eval to behavioral categories (like multi-step retrieval) and regularly remove evals that are saturated or we no longer feel are useful for the agent + current generation of models.
2. **Split data per category.** Create Optimization and Holdout sets. This is very important! We find that autonomous hill-climbing has a tendency to overfit to tasks so holdout sets ensure that learned optimizations work on previously unseen data, though the general distribution should match existing evals. This mirrors what production will look like.
3. **Run a Baseline.** Run a baseline experiment on the Optimization & Holdout sets before any edits. This grounds all updates in the update steps.
4. **Optimize.** Each iteration runs autonomously with optional human review:
 - **Diagnose** from traces. Scores aggregate performance over categories and then Traces show the details of what went wrong and why.
 - **Experiment** a targeted harness change. We scope to one change at a time to avoid confounding but that may mean updating a prompt and tool simultaneously so the system works well together.
5. **Validate:** In each step, the loop checks to make sure that the proposed change helped pass new evals while avoiding regressions on existing passing cases. It's common that some change results in a net overall score gain with some regressions. The agent gets context of these regressions so it can try to fix them in the next update without losing the gains from the existing update.
6. **Human review.** We manually review changes and edge cases metrics miss. This often includes instructions that are overfit to the optimization set and although they don't hurt generalization, they end up being a waste of tokens. This gives us another sanity check and gate against overfitting.



Examples of harness changes

Here are the kinds of changes the optimization loop can discover and validate:

Prompt and instruction updates. The most common change. The agent keeps misinterpreting a tool's output format, or it's too aggressive about calling a tool when it should ask a clarifying question first. The fix is a targeted instruction update addition like "when querying multiple files that have dependent information, offload information to the filesystem and re-aggregate before giving a final answer."

Adding or updating a tool or tool description. The agent may fail contextualizing when to use a new tool. Edits include examples on of how to use, how to chain this

tool, an updated tool description, and editing the overall tool suite to disambiguate similar tools

Results from the Better-Harness loop

We tested this approach with Claude Sonnet 4.6 and [Z.ai](#)'s GLM-5 on a subset of our evals. **Note:** We have other work underway generalizing Better-Harness across many models in `deepagents` using a bigger eval suite. The goal is to publish a series of **model profiles** that capture the nuances of each model tuned for our evals as a public artifact.

We assembled a small representative sample from existing eval categories and split that sample into a set for hill-climbing and holdout to evaluate generalization. With large or expensive eval sets, we suggest representative/stratified sampling to give a good set to hill-climb against. Once this works well, it can be scaled up to the larger set.

Main experiment goal: discover & fix failure modes over our evals. Port general changes that increase eval performance back to the harness.

We previously observed failure modes such as over-asking follow-up questions and errors in chaining together new tools. After hill climbing on the optimization set, we evaluated the final harness on the holdout using two categories, `tool_selection` and `followup_quality`.

← Go back to blog

Evals are training data for agents

Sourcing good evals

Better-Harness: a recipe for

Results from the Better-Harness loop

Evals maintenance & regressions

The future: automated error detection & fixes

Share

in

X

LangChain

Products ▾ Learn ▾ Docs Company ▾ Prici Try LangSmith

Get a demo

SHARED CHANGE	TASKS OBSERVED IN	MODELS	INSTRUCTI ADDED
Use reasonable defaults	tool_indirect_email_report	Sonnet, GLM-5	reasonable defaults when the request cle implies the
Respect already- fixed constraints	followup_vague_send_report, followup_detailed_calendar_brief	Sonnet, GLM-5	"Do not as for details user alrea supplied."

SHARED CHANGE	TASKS OBSERVED IN	MODELS	INSTRUCTIONS ADDED
Bound exploration before acting	tool_chain_search_then_email	Mostly GLM-5	"Do not keep issuing new duplicate searches c you have enough information draft a con summary."
Ask domain- defining questions first	followup_vague_customer_support, followup_vague_monitor_system	Sonnet, GLM-5	"Ask domain defining questions before implement questions."

For evals that inject new tools into the default harness like `search - then - email` , the loop discovered better descriptions of how to use and compose those tools. This is promising for builders creating vertical agents across domains, because optimization loops adapt well to the task specifics in context.

Evals maintenance & regressions

Along with hill climbing, evals also explicitly capture and protect against regressions over time. Once our agent handles a case correctly, we don't want to lose that gain. The eval becomes a regression test. This is similar to ideas in traditional software engineering like Test Driven Development (TDD). Some regressions are bound to happen across many changes over time so we select a subset of evals that we always want to pass and look at our run suspiciously if these suddenly fail.

We don't think our eval suite should grow monotonically, spring cleaning of evals is good! We regularly assess whether an eval is still useful because of more

The future: automated error detection & fixes

This approach works because **traces give us a dense feedback signal**. Evals benefit from traces to compare across versions and numerically ground which changes contribute to a better score (which should be a good proxy for a better user experience).

Overall, we point agentic compute at traces to:

- **Derive errors automatically.** We want to constantly monitor our agent traces to classify and cluster failures in production.
- **Generate evals from production.** A trace where the agent made a mistake is an eval case. A trace where a user corrected the agent is even better. The flywheel: more usage → more traces → more evals → better harness
- **Compare harness versions.** Side-by-side trace comparisons show what changed in the harness that contributed to new behavior

Every trace contains valuable data to produce a potential eval. And every (good) eval makes the harness better. To facilitate this, all agent runs are logged to [LangSmith](#) with full traces. This gives us trace-level diagnosis for the optimization loop, production monitoring for regression detection, and trace mining for eval generation.

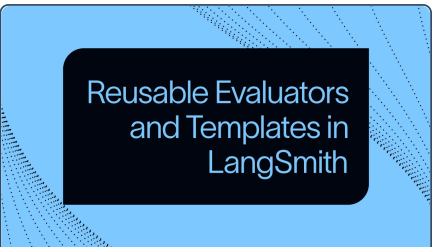
Our main takeaways and ongoing work:

Evals are training data for autonomous harness engineering. The same principles that make ML training work such as data quality, train/test splits, and generalization checks apply to agent development.

Fitting models to harnesses. There's a large amount of work that goes into fitting every model to its harness. For example, the [codex prompting guide](#) suggests a certain format for their `Edit` tool. This requires a bigger search space and eval set, we're excited to share real examples of what that looks like for any team looking to do this.

Overall, tracing and maintaining good evals is what makes this system work in practice. Invest in this early with your team and come build the future of autonomously improving agents. We open sourced a [research version of this scaffold](#) for builders to experiment with.

Related content



Reusable Evaluators and Templates in LangSmith

Observability & Evals

LangSmith

Reusable Evaluators and Evaluator Templates in LangSmith



C. Qiao, J. Talbot
April 16, 2026

4 min



Human Judgment in the Agent Improvement Loop

LangSmith

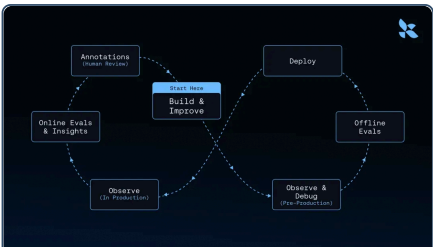
Observability & Evals

Human judgment in the agent improvement loop



Rahul Verma
April 9, 2026

11 min



The agent improvement loop starts with a trace

Conceptual Guide

Observability & Evals

LangSmith

The agent improvement loop starts with a trace



Sam Crowder
March 31, 2026

12 min



Sign up for our newsletter to stay up to date

john.

Subscribe

See what your agent is really doing

LangSmith, our agent engineering platform, helps developers debug every agent decision, eval changes, and deploy in one click.

Try LangSmith

Get a demo

Products

LangSmith Platform
LangSmith Observability
LangSmith Evaluation
LangSmith Deployment
LangSmith Fleet
Deep Agents
LangChain
LangGraph

Resources

Blog
Customer Stories
Guides
LangChain Academy
Community
Changelog
Docs
Support

Company

About
Careers
Partners
Trust Center
Marketing Assets
Events

Sign up for our newsletter to stay up to date

Your email

Subscribe



LangChain

All systems operational

Privacy policy Terms of service